SCHRITT 42 - PERSISTED QUERIES / TRUSTED DOCUMENTS

Deutsche Fassung - konkrete Umsetzung + Code-Zusammenfassung

1. Ziel

Der Production-Client sendet keine beliebigen GraphQL-Dokumente. Operationen werden vorher im OperationStore registriert; zur Laufzeit werden operationId + variables gesendet. Trusted Documents bilden eine Allowlist der erlaubten Operationen.

2. Voraussetzung

GraphQL::Pro::OperationStore gehört zu GraphQL Pro. Wenn das Projekt GraphQL Pro noch nicht verwendet, ist dieser Schritt als Production-Hardening zu verstehen; kein eigenes StoredQuery-System improvisieren.

3. Schema konfigurieren

OperationStore zum Schema hinzufügen. Falls Tracing/APM verwendet wird, sollte das Tracing vor OperationStore konfiguriert werden.

```
class YourAppSchema < GraphQL::Schema
  query Types::QueryType
  mutation Types::MutationType

  use GraphQL::Dataloader

  # Tracing/APM (falls vorhanden) vor OperationStore
  use GraphQL::Pro::OperationStore
end
```

4. Persistenz-Backend

OperationStore kann ActiveRecord oder Redis verwenden. Für Rails + PostgreSQL ist ActiveRecord zum Lernen übersichtlich. Das Tabellenschema nicht selbst erraten, sondern Generator/Dokumentation passend zur installierten GraphQL-Pro-Version verwenden.

5. Routes

GraphQL-Endpoint, OperationStore Sync API und Dashboard bereitstellen. Das Dashboard in Produktion nur mit Admin-Authentifizierung zugänglich machen.

```
# config/routes.rb
using GraphQL::Pro::Routes

Rails.application.routes.draw do
  post "/graphql", to: "graphql#execute"

  # Produktion: Dashboard mit Admin-Authentifizierung schützen
  mount YourAppSchema.dashboard,
        at: "/graphql/dashboard"

  mount YourAppSchema.operation_store_sync,
        at: "/graphql/sync"
end
```

6. Frontend-Operationen organisieren

Operationen in .graphql-Dateien trennen, z. B. GetProjects.graphql, GetTasks.graphql, CreateTask.graphql und UpdateTask.graphql. Jede Operation sollte einen eindeutigen Operation Name besitzen.

```
query GetTasks(
  $projectId: ID!
  $filter: TaskFilterInput
  $first: Int
  $after: String
) {
  tasks(
    projectId: $projectId
    filter: $filter
    first: $first
    after: $after
  ) {
    nodes {
      id
      title
      status
      priority
```

```
    }

    pageInfo {
      hasNextPage
      endCursor
    }
  }
}
```

7. Operationen synchronisieren

Flow: *.graphql -> Sync Client/CLI -> /graphql/sync -> OperationStore. Nach dem Sync kennt der Server die gültigen Operationen eines Clients wie project-web.

8. Runtime Request

Nach dem Sync sendet der Client operationId und variables. Variables bleiben dynamisch; eine Persisted Query hardcodiert weder projectId noch Filter oder Pagination.

```
{
  "operationId": "project-web/abc123...",
  "variables": {
    "projectId": "10",
    "first": 20
  }
}
```

9. Trusted Documents

Gewünschtes Production-Verhalten: bekannte operationId -> ausführen; unbekannte operationId -> ablehnen; beliebige Raw Query -> nicht ausführen. Development/Test dürfen für GraphiQL und RSpec weiterhin Raw Queries erlauben.

10. Authorization bleibt Pflicht

Eine vertrauenswürdige Operation bedeutet keinen vertrauenswürdigen Benutzer. Eine persistierte DeleteProject-Mutation muss weiterhin authenticate_user! und authorize! project, :destroy ausführen. OperationStore ersetzt weder Policies noch Complexity, Pagination oder Rate Limiting.

11. Logging aus Schritt 41

operation_id kann zum Structured Logging ergänzt werden. operationId niemals zur Autorisierung verwenden und keine Raw Query, Variables oder Tokens loggen.

```
{
  operation: operation_name,
  operation_id: operation_id,
  request_id: request_id,
  user_id: user&.id,
  duration_ms: duration_ms
}
```

12. Minimale RSpec-Tests

Drei Fälle testen: bekannte operationId -> Erfolg; unbekannte operationId -> Ablehnung; bekannte Operation + nicht autorisierter Benutzer -> Policy blockiert weiterhin. Fehlermeldungen erst nach Prüfung der installierten Version fest codieren.

13. Deployment-Reihenfolge

Sichere Reihenfolge: 1) Backend-Schema mit Unterstützung für die neue Operation deployen; 2) Persisted Operations synchronisieren; 3) Frontend deployen. Andernfalls kann eine neue operationId als unbekannt abgelehnt werden.

14. Verbindung zu Schritt 43

OperationStore zeigt, welche Clients welche Operationen registriert haben. Das hilft bei Schema Evolution: Vor dem Entfernen eines deprecated Fields prüfen, ob noch Trusted Operations dieses Field verwenden.

Zusammenfassung - Code

Der wichtigste Code auf einen Blick:

```
class YourAppSchema < GraphQL::Schema
  query Types::QueryType
  mutation Types::MutationType

  use GraphQL::Dataloader
```

```
  # Tracing/APM (falls vorhanden) vor OperationStore
  use GraphQL::Pro::OperationStore
end

# config/routes.rb
using GraphQL::Pro::Routes

Rails.application.routes.draw do
  post "/graphql", to: "graphql#execute"

  # Produktion: Dashboard mit Admin-Authentifizierung schützen
  mount YourAppSchema.dashboard,
        at: "/graphql/dashboard"

  mount YourAppSchema.operation_store_sync,
        at: "/graphql/sync"
end

{
  "operationId": "project-web/abc123...",
  "variables": {
    "projectId": "10",
    "first": 20
  }
}
```

Checkliste

GraphQL Pro/OperationStore ✓ | Persistenz-Backend ✓ | /graphql/sync ✓ | Dashboard nur Admin ✓ | benannte .graphql-Operationen ✓ | Sync vor Frontend-Deploy ✓ | Tests für bekannte/unbekannte IDs ✓ | Policy bleibt Pflicht ✓ | optional operation_id loggen ✓